

Tabla Comparativa: Tipos de Datos en Django Models vs. SQL

Campo de Modelo Django	Tipo de Dato SQL Común	Descripción
CharField	VARCHAR(n)	Para cadenas de texto cortas. Requiere max_length.
TextField	TEXT	Para cadenas de texto largas sin un límite específico.
IntegerField	INTEGER	Para números enteros.
BigIntegerField	BIGINT	Para números enteros de mayor rango (64 bits).
SmallIntegerField	SMALLINT	Para números enteros de rango pequeño.
PositiveIntegerField	INTEGER	Similar a IntegerField, pero con una restricción a nivel de base de datos para que solo acepte valores positivos.
AutoField / BigAutoField	INTEGER / BIGINT con AUTO_INCREMENT	Un campo entero que se autoincrementa automáticamente. Se usa para claves primarias.
BooleanField	BOOLEAN o TINYINT(1)	Para almacenar valores True o False.
DecimalField	DECIMAL(p, s) o NUMERIC(p, s)	Para números de punto fijo con precisión y escala decimal definidas. Ideal para datos financieros.
FloatField	FLOAT o DOUBLE PRECISION	Para números de punto flotante.
DateTimeField	DATE	Para almacenar fechas (año, mes, día).
DateTimeField	DATETIME o TIMESTAMP	Para almacenar fechas y horas.
TimeField	TIME	Para almacenar horas (hora, minuto, segundo).
DurationField	INTERVAL (PostgreSQL) o BIGINT (otros)	Para almacenar periodos de tiempo. Django lo almacena en microsegundos si la BD no tiene un tipo nativo.
EmailField	VARCHAR(254)	Un CharField que valida que el texto sea una dirección de correo electrónico válida.
URLField	VARCHAR(200)	Un CharField que valida que el texto sea una URL válida.

Campo de Modelo Django	Tipo de Dato SQL Común	Descripción
<code>FileField</code> / <code>ImageField</code>	<code>VARCHAR(100)</code>	Almacena la ruta (una cadena de texto) al archivo subido, no el archivo en sí.
<code>BinaryField</code>	<code>BLOB</code> o <code>BYTEA</code>	Para almacenar datos binarios crudos.
<code>UUIDField</code>	<code>UUID</code> (PostgreSQL) o <code>CHAR(32)</code> (otros)	Para almacenar identificadores únicos universales (UUID).
<code>ForeignKey</code>	<code>INTEGER</code> o el tipo de la PK referenciada	Crea una relación "uno a muchos" almacenando la clave primaria (ID) del modelo relacionado.
<code>OneToOneField</code>	<code>INTEGER</code> con <code>UNIQUE</code>	Similar a <code>ForeignKey</code> pero con una restricción <code>UNIQUE</code> para garantizar una relación "uno a uno".
<code>ManyToManyField</code>	(Tabla intermedia)	Django crea una tabla intermedia adicional para gestionar las relaciones "muchos a muchos".
<code>JSONField</code>	<code>JSON</code> o <code>JSONB</code> (PostgreSQL)	Para almacenar datos en formato JSON de manera nativa en la base de datos.